Université Abdelmalek Essaâdi
École nationale des sciences appliquées
d'Al Hoceima

Fliére: Transformation Digitale et IA

Module: Deep Learning

Compte Rendu:

TP2(MLP)

Réalisé par:
Amal Dadda Amami

Encadré par:
Prof.M.Cherradi

Année Universitaire:
2025/2026

1. Objectif du TP

L'objectif de ce TP est de comprendre et implémenter un Perceptron Multicouche pour résoudre différents types de problèmes :

- Classification d'images (MNIST, CIFAR-10)
- Classification binaire (Breast Cancer)
- Régression (Boston Housing)

2. Outils utilisés

TensorFlow/Keras, Scikit-learn et NumPy

3. Lien Colab:

https://colab.research.google.com/drive/1-aw3UjJybof2iv6hGmnw8QkPL3EosqNb?usp=sharing

Exercice 1 : Classification MNIST

Le dataset MNIST contient des images de chiffres manuscrits (0 à 9).

1- Prétraitement

- Transformation des images (28×28) en vecteurs de taille 784
- Normalisation des pixels entre 0 et 1
- Transformation des labels avec one-hot encoding

```
#on transform chaque matrice (28x28) en vecteur de taille(784,) et normalisation
x_train = x_train.reshape(-1, 784) / 255.0
x_test = x_test.reshape(-1, 784) / 255.0

#pour transformer une étiquette en vecteur binaire on fait:
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)
```

2- Architecture du modèle

- Entrée : 784 neurones
- Couche cachée 1 : 128 neurones (ReLU)
- Couche cachée 2 : 64 neurones (ReLU)
- Sortie : 10 neurones (Softmax)

```
model = models.Sequential([
    layers.Dense(128, activation='relu', input_shape=(784,)),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

3- Résultats

- Bonne précision
- Le modèle apprend rapidement même avec peu d'époques

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9762 - loss: 0.0779
[0.07794612646102905, 0.9761999845504761]
```

Exercice 1 : Classification MNIST

Dataset d'images couleur (32×32) réparties en 10 classes.

1- Prétraitement

- Flatten des images (3072 features)
- Normalisation
- One-hot encoding

```
x_train = x_train.reshape(-1, 3072) / 255.0
x_test = x_test.reshape(-1, 3072) / 255.0

y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)
```

2- Architecture du modèle

- Entrée : 3072 neurones
- 512 neurones (ReLU)
- 256 neurones (ReLU)
- Sortie : 10 neurones (Softmax)

```
model = models.Sequential([
    layers.Dense(512, activation='relu', input_shape=(3072,)),
    layers.Dense(256, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

3- Résultats

- Précision plus faible que MNIST
- Difficulté due à la complexité des images

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.4855 - loss: 1.4533
[1.453277587890625, 0.4855000078678131]
```

Exercice 3 : Breast Cancer (Classification binaire)

Classification des tumeurs en 0 : bénigne et 1 : maligne

1- Prétraitement

- Normalisation des données
- Séparation train/test

```
# Normalisation
scaler = StandardScaler()
X = scaler.fit_transform(X)
```

```
# Sparation 80% train et 20% test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

2- Architecture du modèle

- Entrée : 30 neurones
- 64 neurones (ReLU)
- 32 neurones (ReLU)
- Sortie : 1 neurone (Sigmoid)

```
model = models.Sequential([
    layers.Dense(64, activation='relu', input_shape=(30,)),
    layers.Dense(32, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])
```

3- Résultats

- Précision élevée (> 95%)
- Modèle très performant sur données tabulaires

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.9649 - loss: 0.1029
[0.10285323858261108, 0.9649122953414917]
```

Exercice 4 : Régression Boston Housing

Prédiction du prix des maisons.

1- Prétraitement

- Normalisation des variables

```
# Normalisation
scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)
```

2- Architecture du modèle

- Entrée : 13 neurones
- 64 neurones (ReLU)
- 32 neurones (ReLU)
- Sortie : 1 neurone

```
model = models.Sequential([
    layers.Dense(64, activation='relu', input_shape=(13,)),
    layers.Dense(32, activation='relu'),
    layers.Dense(1)
])
```

3- Résultats

- Erreur RMSE faible
- Bonne capacité de généralisation

Analyse générale

- Les MLP sont efficaces pour différents types de problèmes
- Les performances dépendent fortement du type de données etdu prétraitement aussi.
- Les images complexes nécessitent souvent des modèles plus avancés

Conclusion

Ce TP m'a permis de :

- Comprendre le fonctionnement des réseaux de neurones
- Implémenter des modèles MLP pour classification et régression
- Observer l'impact de l'architecture et des données sur les performances